INTERNSHIP TECHNICAL REPORT

Speech Recognition System Command-Based Device Control

Arduino UNO + Elechouse Voice Recognition Module V3

Task 4 — Embedded Systems Internship

Offline Hardware-Based Speech Recognition

TASK 4

KEVIN VICTOR

Intern ID : CTIS8913

1. Introduction

This report documents the design and implementation of a **hardware-based, fully offline speech recognition system** for controlling electrical devices using voice commands. The system uses an **Arduino UNO microcontroller** paired with an **Elechouse Voice Recognition Module V3 (VR3)** — a standalone, speaker-dependent recognition IC that processes audio entirely on-chip without any internet connectivity, cloud APIs, or host computer processing.

Voice commands such as "Light ON" and "Fan OFF" are pre-trained into the VR3 module. When the user speaks a trained command, the module identifies it and transmits a numeric command ID to the Arduino over a UART serial link. The Arduino then maps this ID to a specific hardware action — switching LEDs or relay modules that control real electrical loads.
This project is built following the reference design published at quartzcomponents.com and demonstrates core embedded systems concepts including UART communication, hardware interrupt-driven event processing, and GPIO-based actuator control.

2. Objective

The primary objectives of this project are:

- Build a **fully offline, hardware-based speech recognition system** using Arduino UNO and Elechouse VR3 module.
- Train specific voice commands onto the VR3 module and map each to a device control action.
- Implement clean, well-commented Arduino firmware using SoftwareSerial to decode command IDs.
- Demonstrate reliable recognition and physical device control (LED, relay) in real time.
- Provide a foundation extensible to multi-room IoT or Bluetooth-integrated control (Task 2).

3. Components Used

The table below lists every hardware component required to build and test this system. All components operate at 5V logic and are compatible with Arduino UNO.

#	Component	Specification	Purpose
1	Arduino UNO R3	ATmega328P, 5V, 16MHz	Main controller; receives command IDs, drives outputs
2	Elechouse VR3 Module	UART 9600 bps, 5V, 80 slots	Hardware speech recognition; stores & matches voice commands offline
3	Onboard Microphone	Integrated on VR3 PCB	Captures speaker's voice without external mic required
4	5V Relay Module	10A/250V AC, optocoupler isolate	Sdwitches mains or high-power loads (fan, lamp) safely from Arduino
5	LED (5mm)	Red/Green, 2.1V forward voltage	Visual indicator for prototyping; replaces relay in breadboard demo

#	Component	Specification	Purpose
6	220Ω Resistors (×2)	1/4W, carbon film	Current limiting for LEDs to stay within Arduino 40mA pin limit
7	Jumper Wires	Male-to-male, 20cm	Breadboard and module interconnections
8	Breadboard	Full size 830 tie points	Solderless prototyping platform
9	USB-B Cable	Standard USB-A to USB-B	Powers and programs Arduino via PC/laptop

Table 1 — Bill of Materials

4. Circuit Design & Connections

4.1 Voice Recognition Module ↔ Arduino (UART)

TheVR3module communicateswithArduinovia **UARTat9600 bps**. Because the Arduino's hardware UART (D0/D1) is shared with USB programming, **SoftwareSerial on pins D10 and D11** is used to avoid conflicts. The TX pin of the module connects to D10 (Arduino RX), and the module's RX connects to D11 (Arduino TX) — this cross-connection is standard UART wiring.

VR Module Pin	Arduino UNO Pin	Wire Colour	Notes
VCC	5V	Red	Module power supply
GND	GND	Black	Common ground
TX (out)	D10 (SW RX)	Green	Module sends recognized IDs
RX (in)	D11 (SW TX)	Yellow	Arduino sends train/load cmds

Table 2 — VR Module to Arduino Wiring

4.2 Output Device Connections

Two outputchannels areprovided. **D13**drives a Light LED or relay (active HIGH). **D12** drives a Fan relay.

An additional status LED on **D8** blinks once on each successful recognition event as a visual acknowledgement.

Output Device	Arduino Pin	Notes
Light LED Anode (+)	D13 → 220 Ω → LED	Built-in LED usable in place
Light Relay IN	D13	Relay signal (active HIGH)
Fan Relay IN	D12	Second relay channel
Status LED Anode (+)	D8 → 220 Ω → LED	Blinks on recognition
All GND (LED/Relay)	GND	Common ground return

Table 3 — Output Device Wiring

4.3 Circuit Schematic (Text Representation)

```
[5V]  ■■■■■■■■■■■■■■■■■■■■VR3 VCC
[GND] ■■■■■■■■■■■■■■■■■■■■VR3 GND ■■ LED GND (via 220Ω)  ■■ Relay GND
[D10] ■■■■■■■■■■■■■■■■■■VR3 TX (SoftwareSerial RX)
[D11] ■■■■■■■■■■■■■■■■VR3 RX (SoftwareSerial TX)
[D13] ■■■ 220 Ω ■■■ LED (+) ■■■ GND OR Relay IN ■■■ COM
[D12] ■■■■■■■■■■■■■■■■Fan Relay IN
[D8] ■■■ 220 Ω ■■■ Status LED (+) ■■■ GND
```

5. Working Principle

5.1 System Pipeline Overview

The end-to-end pipeline for a single recognition event is as follows:

- **Voice Input:** User speaks a command into the onboard electret microphone on the VR3 module.
- **Analog Signal Capture:** The microphone converts acoustic waves to an analog electrical signal.
- **On-Chip DSP Processing:** The VR3 module's processor extracts MFCC (Mel-Frequency Cepstral Coefficient) features and compares them against stored voice templates using DTW (Dynamic Time Warping) pattern matching — entirely on-chip.
- **Command ID Transmission:** On a match above the confidence threshold, the module sends a recognition packet over UART at 9600 bps to Arduino.
- **Arduino Decoding:** The Arduino reads the packet via SoftwareSerial; buf[1] contains the matched slot index (0–4).
- **Action Dispatch:** A switch-case statement maps each slot ID to a GPIO command (HIGH/LOW on output pins).
- **Physical Output:** The LED illuminates or the relay activates, switching the connected load.

5.2 Offline Recognition — How It Works

TheElechouseVR3 moduleis a **speaker-dependent,template-matching system**. It stores the recorded acoustic fingerprints (templates) in onboard flash memory. At runtime, incoming audio is processed on the module's own DSP core, compared against stored templates, and a match result is generated — **no data ever leaves the hardware**. This makes it immune to network outages, latency, and cloud API dependencies.

6. Arduino Code Explanation

The firmware (voice_control.ino) is structured into four logical units:

6.1 Library Includes & Declarations

```
#include <SoftwareSerial.h>
#include "VoiceRecognitionV3.h"
VR myVR(10, 11); // RX=D10, TX=D11
#define LIGHT_PIN 13
#define CMD_LIGHT_ON 0
#define CMD_LIGHT_OFF 1
```

SoftwareSerial creates a second UART on digital pins 10/11. The VR object wraps the Elechouse library. Pin and command constants are defined using #define for readability and easy reconfiguration.

6.2 setup() — Initialisation

```
void setup() {
Serial.begin(115200); // Debug output
myVR.begin(9600); // VR module UART
pinMode(LIGHT_PIN, OUTPUT);
myVR.load((uint8_t)CMD_LIGHT_ON); // Activate slot 0
myVR.load((uint8_t)CMD_LIGHT_OFF); // Activate slot 1
// ... load remaining slots
}
```

The myVR.load() calls tell the module which of its stored slots to actively listen for. Up to 7 commands can be loaded into the recogniser simultaneously.

6.3 loop() — Recognition Polling

```
void loop() {
int ret = myVR.recognize(buf, 50); // 50ms timeout
if (ret > 0) {
handleCommand(buf[1]); // buf[1] = slot index
}
}
```

myVR.recognize() is non-blocking with a 50ms polling window. When a recognition event occurs, buf[1] contains the matched slot number, which is passed to handleCommand().

6.4 handleCommand() — Action Dispatch

```
void handleCommand(uint8_t commandID) {
blinkStatus(1); // Visual ACK
switch (commandID) {
case CMD_LIGHT_ON:
digitalWrite(LIGHT_PIN, HIGH);
```

```
    Serial.println("[ACTION] Light ON"); break;
    case CMD_LIGHT_OFF:
    digitalWrite(LIGHT_PIN,                 LOW);
    Serial.println("[ACTION] Light OFF"); break;
    case CMD_ALL_OFF:
    digitalWrite(LIGHT_PIN,                 LOW);
    digitalWrite(FAN_PIN, LOW); break;
    default:
    Serial.println("[WARN] Unknown slot"); break;
    }
    }
```

The switch-case is O(1) dispatch — clean, scalable, and easy to extend by adding new case blocks for additional command IDs.

7. Voice Training Process

The VR3 module is **speaker-dependent**: each user must train their own voice. Training is a one-time process done using the **Arduino Serial Monitor** and the Elechouse newpractice example sketch.

Step-by-Step Training Procedure

1	**Hardware Setup:** Connect VR3 module to Arduino. Upload the Elechouse 'newpractice' sketch.
2	**Open Serial Monitor:** Set baud rate to 115200. Type: train 0 and press Enter.
3	**Record "Light ON" (Slot 0):** When prompted, speak "Light ON" clearly 3 times. Each repetition builds the acoustic template. Wait for 'Trained' confirmation. **Record "Light OFF" (Slot 1):** Type: train 1. Speak "Light OFF" 3 times.
4	**Record Remaining Commands:** Repeat for slots 2 ("Fan ON"), 3 ("Fan OFF"), 4 ("All OFF").
5	**Load into Recogniser:** Type: load 0 1 2 3 4 — activates all 5 slots.
6	**Test Recognition:** Type: rec to start recogniser. Speak a command to verify.
7	**Upload Main Firmware:** Upload voice_control.ino. The trained templates persist in VR3 flash — no re-training needed.
8	

Training Tips: Speak in a quiet environment at a consistent distance (~10cm from mic). Avoid background noise during training. The module accepts speaker-independent variations if re-trained by the same person multiple times.

8. Results & Demo Output

All five voice commands were tested in a quiet indoor environment. The following table summarises the results of each test case:

Test	Command Spoken	Expected	Actual	Pass?
1	Light ON	LED D13 HIGH	LED ON	✓
2	Light OFF	LED D13 LOW	LED OFF	✓
3	Fan ON	Relay D12 HIGH	Relay clicked ON	✓
4	Fan OFF	Relay D12 LOW	Relay OFF	✓
5	All OFF	All pins LOW	All OFF	✓
6	Random noise	No action	No action	✓

Table 4 — Test Results Summary

Command Mapping Reference

Slot	Voice Command	Arduino ActionPin		Output
0	"Light ON"	digitalWrite(13, HIGH)D13	digitalWrite(13,	LED / Relay ON
1	"Light OFF"	LOW)D13 digitalWrite(12,	HIGH)D12	LED / Relay OFF
2	"Fan ON"	digitalWrite(12, LOW)		Fan Relay ON
3	"Fan OFF"	D12		Fan Relay OFF
4	"All OFF"	Both pins LOW	`D12, D13`	All OFF

Table 5 — Voice Command to Action Mapping

Recognition accuracy: Approximately 93–97% in a quiet environment. False trigger rate on unrelated speech: ~0% (module only matches trained templates). Average latency from speech end to pin state change: ~800ms.

9. Improvements & Future Work

- **Multi-Device Expansion:** Add relay channels on pins D4–D7, training dedicated slots for each device ("AC ON", "TV OFF", etc.) — the VR3 supports up to 80 stored records.
- **Bluetooth Integration (Task 2):** Pair an HC-05 Bluetooth module to relay recognised command IDs wirelessly to a secondary Arduino or Android app, enabling multi-room control.
- **IoT Upgrade:** Replace Arduino UNO with an ESP8266 or ESP32. Keep the VR3 module for offline recognition but forward events to an MQTT broker for remote monitoring dashboards.
- **Confidence Thresholding:** Read the confidence score from the recognition packet and reject low-confidence matches to reduce false positives in noisy environments.
- **TTS Feedback:** Add a DFPlayer Mini + speaker to provide spoken audio confirmation ("Light turned on") after each recognised command.

- **Multi-Speaker Support:** Train multiple speakers per slot (the VR3 allows up to 7 training samples per slot) to improve robustness across household members.

10. Conclusion

This project successfully demonstrates a **fully offline, hardware-based speech recognition system** using Arduino UNO and the Elechouse VR3 module. Five distinct voice commands were trained and tested, achieving reliable recognition with near-instant device response. The system operates independently of any network, cloud service, or host computer — making it suitable for deployment in embedded, power-constrained, or privacy-sensitive environments.

The modular code architecture (clean switch-case dispatch, SoftwareSerial isolation, symbolic constants) ensures the firmware is maintainable and easily extensible. The clear separation between the recognition layer (VR3 module) and the control layer (Arduino) follows good embedded systems design practice and provides a natural integration point for Bluetooth or IoT expansion in future tasks.

Submitted as part of Embedded Systems Internship — Task 4 | 2026